ACTI 4850 – Lab 7 – Static Code Analysis

Instructor	Mike Mulder (mmulder10@bcit.ca)
Total Marks	10
Due Dates	Before End of Next Class (After the Midterm Week): • March 3rd for Sets B and C • March 4th for Set A

Applicable Requirements

- **(New) REQ1180 –** The Enterprise Development Environment shall support a Static Code Analysis capability.
- **(New) REQ1190 –** The Enterprise Development Environment shall support Continuous Integration Pipelines for Java projects. At minimum, the pipeline will include build, test, analysis, packaging and artifact storage. Note: **We are adding build, analysis and test stage for a Java Maven project today.**

Group Work

You will be working on the same Azure cloud environment as the previous lab, shared with your Lab partner. This lab will be done together with your partner.

Part 1 – Install SonarQube

Create a Linux Virtual Machine (using the Virtual Machines Service) with the following specifications under your Azure for Students subscription:

- Resource group name: SonarQube (note: we may also install additional software here in a couple weeks)
- Virtual machine name: SonarQube
- Region: (US) East US (or another region where a B2ms is available)
- Image: Ubuntu 22.04 LTS (or later)
- Size: B2ms
- Authentication Type: SSH public key (create an SSH key if necessary and paste in the public key)
- Inbound Ports: Allow SSH (22) only

Leave everything else at the defaults and Create the Virtual Machine. This may take a few minutes.

Create a DNS Name for your "SonarQube" Virtual Machine that is named similar to:

sonarqube-acit4850-group<group #>.<region>.cloudapp.azure.com

Note: We will also be using this server for Artifact Management hence the generic name.

Add an inbound security rule that allows access to your SonarQube Virtual Machine on TCP port 9000.

SSH to your new VM and install Docker:

- https://docs.docker.com/engine/install/ubuntu/

Manage Docker as a Non-Root User:

- https://docs.docker.com/engine/install/linux-postinstall/

Run the SonarQube Requirements on the VM based on the DockerHub documentation:

- https://hub.docker.com/_/sonarqube/

Create the docker-compose.yml file based on this site:

- https://github.com/SonarSource/docker-sonarqube/blob/master/example-compose-files/sq-with-postgres/docker-compose.yml
- **Change the SonarQube image version to 8.7-community**
- This will create a SonarQube container with a PostgreSQL database
- Add the following environment variable on the sonarqube container to set the context path to /sonarqube
    - SONAR_WEB_CONTEXT: /sonarqube
    - Note: See https://docs.sonarqube.org/9.6/setup-and-upgrade/configure-and-operate-a-server/environment-variables/ for a reference on the SonarQube environment variables
- Update the docker-compose.yml to use environment variables for the credentials (as per ACIT 4630)
- Bring up SonarQube and PostgreSQL with docker compose up -d
- You should be able to access it at http://<SonarQube DNS>:9000/sonarqube
- Add SonarQube to your Reverse Proxy – https://docs.sonarsource.com/sonarqube-server/8.9/setup-and-upgrade/configure-and-operate-a-server/operating-the-server/ (Refer to section Using an Apache Proxy). All you need to do is to add the ProxyPass and ProxyPassReverse to your default-ssl.conf file and restart apache2
    - Restrict access to SonarQube so it's only through your reverse proxy (may check during the demo).
    - You should not be able to access SonarQube at https://<SonarQube DNS>:9000/sonarqube

<u>Part 2 – Add SonarQube Jenkins Plugin and Maven to Docker Image</u>

Add the Plugin:

- Login to your Jenkins server
- Go to Manage Jenkins
- Go to Manage Plugins
- Select the Available tab
- Find the "SonarQube Scanner" plugin
- Select and install this plugin

Configure the Plugin:

- Go to Manage Jenkins
- Select Configure System
- Find the SonarQube servers section of the configuration

- Check the Environment variables checkbox
- Add A SonarQube server
- Enter a SonarQube installation name. Use 'SonarQube'
- Enter the URL of your SonarQube server
- Save the configuration

Part 3 – Create a New Jenkins Java Agent and Docker Compose to Start Them Together

On your Jenkins VM:

- Create a new dockerfile for your Java Agent with the following content:

```
FROM jenkins/ssh-agent:debian-jdk21 AS ssh-agent
LABEL maintainer="mmulder10@bcit.ca"

# Install selected extensions and other stuff
RUN apt-get update && apt-get -y --no-install-recommends install && apt-get clean

# Install maven
RUN apt-get install -y maven
```

  Note that Maven is a Java build tool needed to build and test our sample Java application.

- Build the Java Agent image with the tag **java-agent**
- Re-build the Python Agent image with the tag **python-agent**
- Download the docker-compose.yml file from the Learning Hub (Content -> Lesson 7) to the VM.
- Update the SSH public key for the Python and Java agent services.
- Stop and remove your existing Jenkins containers
- Bring up all three containers with docker compose: docker compose up -d
- Make sure all three services are running

On your Jenkins UI:

- Go to Manage Jenkins -> Nodes
- Update the Configuration of your Python Agent as follows:
  - python-agent as the name
- Create a new Java Agent (based on the Python Agent) called java-agent and update:
  - The SSH port to match the docker-compose.yml file
  - The label should be **java_agent**
  - On the Jenkins VM, you will need to open up an inbound port rule for the SSH port

Part 4 – Add a Java Build Job and Integrate with Jenkins

Create a new GitLab repo with sample Java code:

- Download the Sample Code from the D2L Course Content -> Week 8
- Create a new Project in your GitLab with this code called JavaSample

- Note that there already is a ci/Jenkinsfile stubbed out for this job

Create a new Jenkins pipeline build job for the sample Java code:

- Call the job JavaSamplePipeline using the stubbed out ci/Jenkinsfile
- Configure the job as before and make sure it successfully runs (note: there is a single build stage)
- The Jenkinsfile already has a build stage. Add the following stages to the Jenkinsfile for this project:

```
stage('Test') {
   steps {
       sh 'mvn test'
   }
   post {
       always {
           junit 'target/surefire-reports/*.xml'
       }
   }
}
stage('Deliver') {
   steps {
       sh './ci/scripts/deliver.sh'
   }
}
```

  Note that the Test stage runs Java junit tests and the Deliver stage deploys the built Java artifact to the local Maven repository (i.e., .m2 folder) and runs it (i.e., simple Continuous Delivery).

<u>Part 5 – Integrate Build Pipeline with SonarQube</u>

Assuming you have configured the SonarQube plugin correctly in Jenkins, the build job should be able to publish code analysis results.

Update your Java build stage as follows to generate and publish static code analysis results to SonarQube:

```
stage('Build') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh 'mvn -B -DskipTests clean package sonar:sonar'
        }
    }
}
```

Add the following stage to fail the build pipeline if the results do not pass SonarQube's quality gate:

```
stage("Quality Gate") {
    steps {
        timeout(time: 1, unit: 'HOURS') {
           waitForQualityGate abortPipeline: true
        }
    }
```

}

Your pipeline stages should be: Build, Quality Gate, Test and Deliver (in that order)

For the above to work you also need to create a webhook to Jenkins in SonarQube.
Login to SonarQube as an Admin (admin/admin is the default).
Goto Administration -> Configuration -> Webhooks.
Add a Webhook to Jenkins with a name and the following URL: https://<Apache DNS>/jenkins/sonarqube-webhook
You do not need a secret unless you've configured Webhook Secret in the Jenkins SonarQube configuration.

Also, we need to enable anonymous access to SonarQube:
Still logged in as Admin, got to Administration -> Configuration -> General Settings -> Security
Disable the Force user authentication option and Save

As per the marking criteria below, make sure the analysis results are published to SonarQube AND verify that the following scenarios cause the build job to fail:

- Invalid Java Syntax (fails on build stage)
- SonarQube quality gate (fails on quality gate stage).
- Failing test (fails on test stage)

Make sure you shutdown any Azure resources (i.e., the VM) to conserve your credits and free tier usage.

Demo, Grading and Submission

A demo of your lab against the applicable requirements which will determine your grade on the lab. All mandatory requirements must be met otherwise you will receive zero on the lab. You can re-demo the lab if you haven't met the mandatory requirements, up to the last class before the midterm week, but you will lose 20% every week late.

Reqt.	Mandatory	Demo	Marks
REQ1180	Yes	• Show your SonarQube installation running behind your reverse proxy (and not directly on the SonarQube VM) • It is running with Docker-Compose and is using environment variables for credentials • Login as the default admin user (with the non-default password)	2
REQ1190	Yes	Java Build Job (4 marks) • Demo the successful running of your Java build job. All stages – build, analysis, test and archive are successful. • Show your Jenkinsfile with the corresponding stages in your GitLab repo.	8

		Build Failures (2 marks)
		• Show that your build job fails in the following cases (note: you'll need to create these scenarios by modifying your codebase):
		  ○ Invalid Java Syntax (fails on build stage)
		  ○ SonarQube vulnerability (fails on analysis stage)
		  ○ Failing test (fails on test stage)
		SonarQube Analysis (2 marks)
		• Show the SonarQube analysis results for your Java build job in SonarQube – show each of the results (Issues, Code Smells, and Vulnerablities).	

REQ1170	Yes	N/A	N/A

<u>Submit your Jenkinsfile file to the Lab 7 dropbox on D2L to receive the marks for your demo.</u>